SENIOR PROJECT · DTETI UGM

MindMeter

Development Plan

How backend, frontend and design stay connected from session 6 to the final demo.

The team

- **Nalen** — Backend · Cloud · AI
- **Christania** — Frontend · Software Engineering
- **Arya** — UI/UX · Group lead

The target

Stage 1 live on Azure by session 8.

Stage 2 (NLP) by session 10. Code freeze at session 11.

Based on Final PRD v1.0 — this plan decides HOW we build it.

READ THIS FIRST

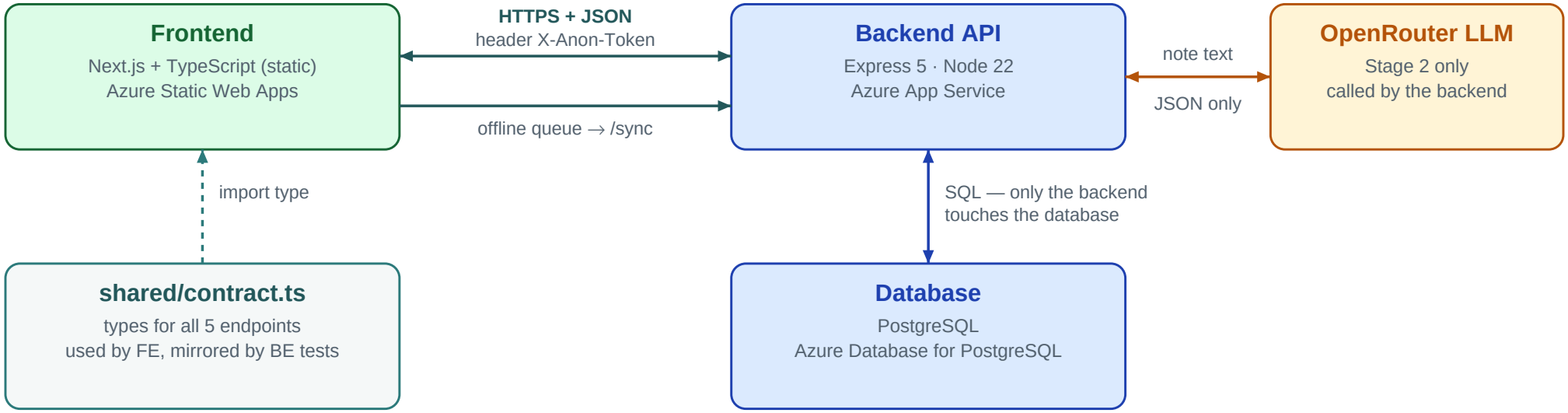
Five rules so our code merges cleanly

Merge problems happen when the frontend and backend each guess what the other sends. These rules make sure there is nothing to guess.

1	**One contract file**	`shared/contract.ts` holds the exact types of every request, response and error. The frontend imports it; a wrong field name is a red error in the editor.
2	**One API file**	Only `frontend/src/lib/api.ts` talks to the backend. Components never call `fetch` themselves.
3	**Contract changes travel together**	Changing the API = changing contract.ts + backend code + backend tests in ONE PR, labelled **contract-change**, approved by Nalen and Christania.
4	**CI is the gate**	Backend: lint + 38 tests on a real database. Frontend: lint + type check + build. A red PR cannot be merged.
5	**Fixed storage keys**	`mindmeter_anon_token` (random UUID) and `mindmeter_sync_queue` (offline check-ins) in localStorage. Never renamed.

TECHNICAL ARCHITECTURE

Three services, one contract



Networking · 20%

Separate client and server talking over HTTPS, plus offline sync that re-sends safely.

Cloud · 20%

Three Azure services working together: Static Web Apps, App Service, PostgreSQL.

AI · 20%

Rule-based trend detection now; OpenRouter sentiment + themes in Stage 2.

TECH STACK

What each part is built with

Layer	Technology	Hosted on	Owner
Frontend	Next.js (App Router) · TypeScript · Tailwind CSS · Recharts · static export	Azure Static Web Apps	Christania
API contract	shared/contract.ts — TypeScript types only, no runtime code	in the repo	Nalen + Christania
Backend	Node 22 · Express 5 · JavaScript · node:test · ESLint	Azure App Service	Nalen
Database	PostgreSQL · backend/src/models/schema.sql · npm run db:migrate	Azure Database for PostgreSQL	Nalen
AI — Stage 1	Rule-based trend (7-day average, low streaks) in backend/src/logic	inside the API	Nalen
AI — Stage 2	OpenRouter LLM, server-side, JSON-only reply, off until configured	OpenRouter	Nalen
Design	Figma: tokens, every screen state, Indonesian microcopy	Figma	Arya
CI / CD	GitHub Actions: lint, tests, type check, build, deploy on main	GitHub	Nalen (BE) · Christania (FE)

Repository layout

```
Senior-Project/
|-- backend/ Express API — done, tested
|-- frontend/ Next.js app — Christania, starts session 6
|-- shared/contract.ts the single source of truth for the API
|-- docs/ PRD, ERD, this plan, usage + demo guides
`-- .github/workflows/ci.yml checks every PR
```

THE CONTRACT

The API cheat sheet (frozen)

Method + path	Token	Request body	Response
`POST /api/checkins`	yes	`{ mood_value: 1–5, note_text?: string ≤200, client_ts?: ISO }`	`{ id, saved, insight?, crisis_flag }`
`POST /api/checkins/sync`	yes	`[{ mood_value, note_text?, client_ts }] (max 50)`	`[{ client_ts, id, saved, crisis_flag }]`
`GET /api/checkins`	yes	—	`[{ id, mood_value, note_text, created_at }]`
`GET /api/insights`	yes	—	`{ trend: stable | declining | improving, message, themes[], resources[] }`
`GET /api/resources`	**no**	—	`[{ id, title, description, url, tags[] }]` — crisis contact first

Always send
Header `X-Anon-Token: <uuid>` from `localStorage['mindmeter_anon_token']`. No token → `401 ANON_TOKEN_REQUIRED`.

Errors always look like
`{ "error": { "code": "...", "message": "..." } }`

Behaviour the frontend relies on
- `crisis_flag: true` → show the support screen immediately. It is the **only** trigger, decided by the backend.
- One check-in per WIB day: a second one updates it.
- Offline: queue items with `client_ts`; re-sending is safe; `saved: true` = remove from queue.
- Phone contacts arrive as `tel:` links in `url`.

WHO DOES WHAT

Three people, three clear lanes

Nalen

Backend · Cloud · AI

- **First (session 6):** add `shared/contract.ts` and a `docker-compose.yml` so the API runs locally with one command.
- Create Azure resources: resource group, PostgreSQL, App Service. Run migrate + demo seed.
- Make the CI deploy job real; set `CORS_ORIGIN` to the frontend URL.
- Load the crisis keyword list and verified UGM contacts.
- Stage 2: verify the OpenRouter model, test Indonesian notes, switch NLP on.

Christania

Frontend · Software Engineering

- Scaffold Next.js in `frontend/`; write `lib/api.ts`, `lib/anonToken.ts`, `lib/syncQueue.ts`.
- Build pages in safety-first order: **support** → **check-in** → **dashboard** → **landing**.
- Every screen handles loading, empty, error and offline states.
- Frontend CI job (lint, type check, build) + Static Web Apps deploy.
- Run the end-to-end test checklist; write the **usage guide** and **demo guide**.

Arya

UI/UX · Group lead

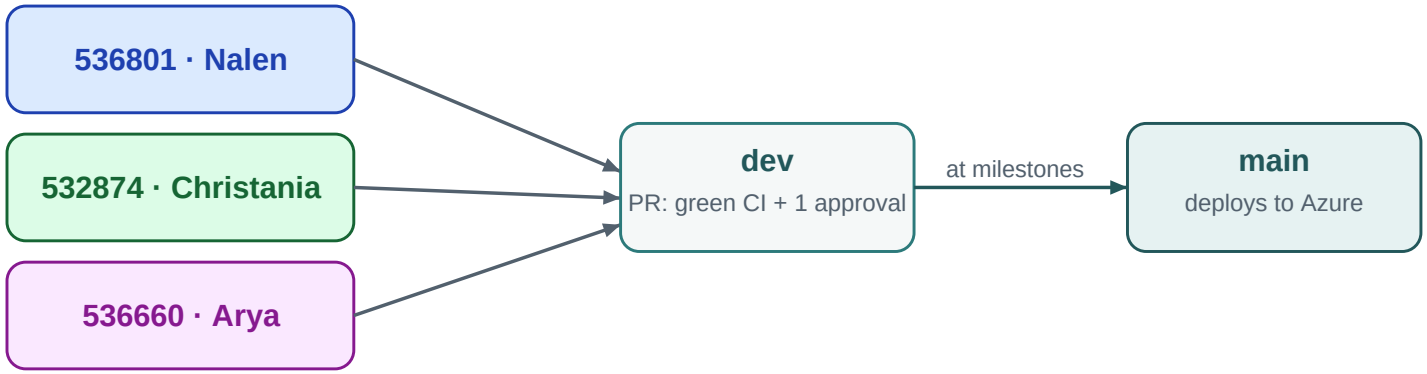
- Design tokens (colours, fonts, spacing) handed over as Tailwind values.
- One Figma frame per screen with **every state**, one screen ahead of the build.
- Indonesian microcopy: calm, non-clinical, with a clear “not a diagnostic tool” line.
- Lead writing the crisis keyword list with the team.
- Keep the Kanban board current; own the demo script and slides.

STAYING CONNECTED

Handoffs and how code gets merged

From → To	What is handed over	Where
Nalen → Christania	contract.ts, local API (docker-compose), deployed API URL, demo token	PR + backend README
Arya → Christania	Figma frame per screen with all states, before she starts that screen	Figma link in the Kanban card
Christania → Nalen	Bug report: the request sent, the response received, what was expected	GitHub issue
Everyone → everyone	Every change goes through a PR — this is also our Git documentation grade	GitHub

Git flow



Commit messages start with `feat:` `fix:` `docs:` `test:`

A card is DONE when it is merged to `dev`, CI is green, and it works on the deployed site.

TIMELINE

Sessions 6 to 12

Session	Nalen — Backend	Christania — Frontend + SE	Arya — UI/UX
6	contract.ts + docker-compose; create Azure resources	Scaffold, API/token/queue libs, **support screen**	Tokens; support + check-in screens; lead keyword list
7	Deploy API; load keywords + verified contacts	**Check-in** + offline sync; **SP 2.4 crisis end-to-end test**	Dashboard screen, all states
8	Seed demo on Azure; help integrate	**Dashboard** + landing; deploy to Static Web Apps	Landing page; review built screens
★	**MILESTONE 1 — Stage 1 live on Azure (dev → main)**		
9	Verify OpenRouter model + Indonesian test	Test checklist on Azure; fix bugs	Polish from test feedback
10	Enable NLP; themes in /api/insights	Themes on dashboard; **usage guide**	Demo script draft
★	**MILESTONE 2 — Stage 2 done**		
11	Regression + fixes	Regression; **demo guide**	Presentation slides
★	**MILESTONE 3 — code freeze**		
12	Rehearse the demo; final docs	Rehearse the demo; final docs	Rehearse the demo; final docs

If we fall behind: Stage 2 (NLP) is the stretch goal. Stage 1 already covers networking, cloud and rule-based AI, so sessions 9–10 become extra testing and polish instead.

START HERE

Your checklist for session 6

Nalen

- [ ] Push shared/contract.ts (PR, contract-change label)
- [ ] Add docker-compose.yml: Postgres + migrate + demo seed
- [ ] Create Azure resource group + PostgreSQL + App Service
- [ ] Share the demo token with the team (not in Git)

Christania

- [ ] npx create-next-app in frontend/ (TypeScript, Tailwind, ESLint)
- [ ] Set output: 'export' and NEXT_PUBLIC_API_URL
- [ ] Write lib/api.ts using the types from shared/contract.ts
- [ ] Build the support screen from Arya's frame

Arya

- [ ] Colour, font and spacing tokens as Tailwind values
- [ ] Support/crisis screen (normal + crisis full-screen)
- [ ] Check-in screen (normal, note, saved, offline)
- [ ] Run the crisis keyword list meeting; rename SP 2.2

Before anyone writes screen code: Nalen's contract.ts PR is merged into `dev`. That single file is what keeps the frontend and backend in sync for the rest of the semester.